

OOM-Free Alpamayo via CPU-GPU Memory Swapping for Vision-Language-Action Models

Seungwoo Roh, Huiyeong Kim, and Jong-Chan Kim

Graduate School of Automobile and Mobility, Kookmin University, Korea

Correspondence: jongchank@kookmin.ac.kr

Abstract—End-to-end Vision-Language-Action (VLA) models for autonomous driving unify perception, reasoning, and control in a single neural network, achieving strong driving performance but requiring 20–60 GB of GPU memory—far exceeding the 12–16 GB available on commodity GPUs. We present a framework, which enables memory-efficient VLA inference on VRAM-constrained GPUs through system-level optimization alone, without model modification. Our work proceeds in three stages: (1) Sequential Demand Layering reduces VRAM usage from model-level to layer-level granularity; (2) Pipelined Demand Layering hides parameter transfer time within layer execution time via transfer–compute overlap; and (3) a GPU-Resident Layer Decision Policy, informed by per-module residency benefit analysis, eliminates the residual transfer overhead that pipelining cannot hide. We further propose a performance prediction model that determines the optimal configuration—both the number and placement of resident layers—from a single profiling run with less than 1.3% prediction error across all configurations. Applied to NVIDIA’s Alpamayo-R1-10B (21.52 GB) on an RTX 5070 Ti (16 GB), our work achieves up to $3.55\times$ speedup over Accelerate offloading while maintaining full BF16 precision.

I. INTRODUCTION

End-to-end autonomous driving increasingly adopts Vision-Language-Action (VLA) models that unify perception, reasoning, and control within a single neural network [1]. Models such as Alpamayo [2], RT-2 [3], and DriveVLM [4] demonstrate strong driving performance, but their GPU memory requirements exceed the VRAM of commodity GPUs. This memory gap creates a practical barrier that makes it difficult to iterate on VLA model integration, testing, and deployment without access to high-end hardware.

NVIDIA’s Alpamayo-R1-10B, released in October 2025, requires 21.52 GB of VRAM, which substantially exceeds the 12–16 GB capacity of widely available consumer GPUs. Consequently, inference on such GPUs is impossible on native Linux due to out-of-memory (OOM) errors. Inference on such hardware is therefore feasible only when CPU memory is jointly used with GPU memory. However, existing mechanisms for such joint use introduce prohibitive latency overhead.

One can avoid these issues by using flagship GPUs with ample VRAM such as the RTX 3090 or RTX 5090, or server-grade GPUs such as the H100, A100, or A6000. However, these GPUs are expensive and demand consistently outstrips supply, making procurement difficult even with sufficient budget. In this paper, we present a systematic solution, which

enables optimal-performance Alpamayo inference on affordable, readily available consumer GPUs.

Model compression techniques such as quantization [5], [6] and pruning can reduce model size to address this problem, but these methods fundamentally incur accuracy loss. Therefore, we target memory reduction and latency improvement through system-level optimization alone, without modifying the given model.

Our framework computes the optimal inference time achievable on a given hardware configuration through static profiling of the target model, and proceeds with optimization aimed at approaching this bound as closely as possible. First, rather than the conventional *preloading* approach that loads all layer parameters into GPU VRAM in advance, we employ *Demand Layering* [7], which loads parameters for each layer immediately before inference, thereby reducing VRAM usage. Furthermore, leveraging the structural advantage that the GPU’s Execution Engine and Copy Engine can simultaneously perform parameter transfer and layer inference computation, we employ *pipelined Demand Layering* with a double buffer to maximally hide layer transfer time within layer execution time. However, certain parts of the model cannot achieve full hiding. For such modules, keeping layers GPU-resident trades VRAM cost for reduced inference latency. We propose a policy for deciding which layers to keep resident, theoretically present a methodology for selecting resident layers, and experimentally validate this across multiple hardware platforms and models.

Our contributions are as follows:

- By analyzing the DMA and compute characteristics of the model’s components, we derive closed-form residency benefit expressions and propose an interleaved placement policy that minimizes end-to-end inference time.
- We present a performance prediction model that estimates inference time for arbitrary residency configurations within 1.3% error from profiling data alone.
- We demonstrate $3.55\times$ speedup over baseline (Accelerate offloading) on the RTX 5070 Ti (16 GB) with full BF16 precision, and validate the analysis framework across multiple GPU platforms.

II. BACKGROUND

A. Alpamayo-R1-10B Model Architecture

Alpamayo-R1-10B [2] processes camera images through a 5-stage inference pipeline:

TABLE I: VRAM breakdown of Alpamayo-R1-10B (BF16). Parameters account for 93% of total VRAM.

Component	Params	VRAM	Share
Parameters	~11.08B	~22.16 GB	93%
Vision Encoder	576.4M	1.15 GB	5.2%
VLM Backbone	7,580M	15.17 GB	68.5%
VLM LM Head	637.7M	1.28 GB	5.8%
Expert Decoder	2,280M	4.56 GB	20.6%
Non-parameters	—	~1.5 GB	7%
Total	—	~23.7 GB	100%

The 21.52 GB cited elsewhere is `max_memory_allocated` (RTX 3090); the 23.7 GB total includes alignment and framework buffers.

- 1) **ViT Encoder**: Qwen3-VL Vision Transformer (27 layers, 1152 dimensions).
- 2) **Patch Merger**: Projects $1152 \times 4 \rightarrow 4096$ dimensions.
- 3) **VLM Prefill**: 36-layer Qwen3-VL-8B backbone with Grouped-Query Attention (GQA) (32Q/8KV).
- 4) **VLM Decode**: Autoregressive token generation—accounts for **78.5%** of total inference time.
- 5) **Diffusion-based Action Generation**: Flow matching with 10 Euler steps produces 64 waypoints. The Expert Decoder (36 layers, $d=2048$) serves as the denoiser, called once per Euler step to predict the velocity field.

B. VRAM Decomposition: Parameter Dominance

Parameters constitute 93% of total memory requirements (Table I). This implies that parameter VRAM reduction is the key target. Demand Layering [7] achieves memory savings by targeting precisely these parameters through layer-wise offloading. Since VLA-based models, including Alpamayo, are similarly dominated by parameter memory, this technique is well suited for application. Non-parameter memory accounts for the remaining 7% and comprises the KV cache, activations, and framework buffers. These non-parameter components are not the target of this work.

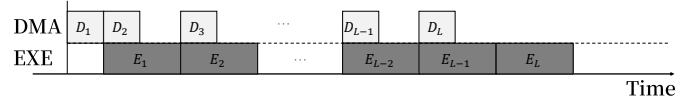
C. PCIe DMA and CUDA Streams

Modern NVIDIA GPUs contain independent Copy Engines and Execution Engines [8]. The CE handles PCIe DMA transfers while the EE executes CUDA kernels. Transfer-compute overlap is achievable by submitting workloads on separate CUDA streams.

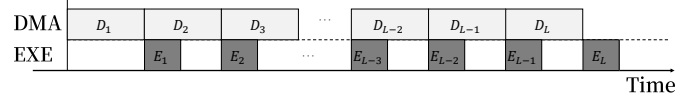
Fig. 1 illustrates transfer-compute overlap. If the PCIe DMA parameter transfer time for a given layer is shorter than the layer execution time, the parameter transfer for layer i can complete before the computation of layer $i-1$ finishes. Let C_{DMA}^i and C_{EXE}^i denote the DMA transfer time and execution time of layer i , respectively. As shown in Fig. 1(a), the total inference time for all L layers is:

$$C_{\text{DMA}}^1 + \sum_{i=1}^L C_{\text{EXE}}^i. \quad (1)$$

If the parameter transfer time exceeds the layer execution time, the situation corresponds to Fig. 1(b). In this case,



(a) $C_{\text{DMA}} < C_{\text{EXE}}$: DMA is fully hidden within the execution window.



(b) $C_{\text{DMA}} > C_{\text{EXE}}$: DMA becomes the bottleneck; pipelining cannot fully hide transfers.

Fig. 1: Inference timelines under two regimes. (a) EXE-intensive pipelining where transfers are fully hidden. (b) DMA-intensive pipelining where transfer overhead is unavoidable.

parameter transfers cannot be fully hidden, and the total inference time becomes:

$$\sum_{i=1}^L C_{\text{DMA}}^i + C_{\text{EXE}}^L. \quad (2)$$

When GPU VRAM is sufficient for all parameters, the model operates in a preloading mode with no runtime parameter loading required, yielding an inference time of $\sum_i C_{\text{EXE}}^i$. Relative to this baseline, the overhead introduced by Demand Layering depends on each layer's characteristics.

To simplify the analysis, we first assume that all layers within a module share the same characteristic—either $C_{\text{DMA}} < C_{\text{EXE}}$ or $C_{\text{DMA}} > C_{\text{EXE}}$ holds uniformly across all layers.

When $C_{\text{DMA}} < C_{\text{EXE}}$, parameter transfers can be fully hidden, incurring only the overhead of the initial layer's transfer time C_{DMA}^1 . In contrast, when $C_{\text{DMA}} > C_{\text{EXE}}$, parameter loading cannot be fully hidden, and each layer incurs an overhead of $C_{\text{DMA}} - C_{\text{EXE}}$. In this case, DMA becomes the bottleneck, pipelining cannot completely hide the transfer, and the only solution is GPU residency to eliminate the transfer entirely.

In practice, a model may contain layers of both types. The total overhead is then a per-layer combination of the two expressions above.

III. PROBLEM ANALYSIS

We identify three challenges in reducing Alpamayo's inference latency: (i) theoretical lower bound, (ii) overhead of existing swapping methods, and (iii) DMA hiding limitation.

A. Theoretical Lower Bound

The RTX 5090 has 21,760 CUDA cores and the RTX 5070 Ti has 8,960 CUDA cores. Therefore, even if the RTX 5070 Ti had 24 GB or more of VRAM, its inference time would still be longer due to the GPU's compute capability. We define the *theoretical lower bound* as the time required to perform Alpamayo inference with infinite VRAM. No offloading strategy can achieve inference latency below this

TABLE II: Measured inference time and theoretical lower bound estimated from per-layer execution times.

GPU	C^{opt}	Measured Time
RTX 5070 Ti	2.86 s	—
RTX 3090	3.13 s	3.12 s

bound. GPUs with ample VRAM can preload all parameters for inference, but for GPUs like the RTX 5070Ti whose VRAM is smaller than the model parameter size, preloading is impossible. We therefore estimate the theoretical lower bound as the simple sum of per-layer execution times:

$$C^{\text{opt}} = \sum_{i=1}^L C_{\text{EXE}}^i. \quad (3)$$

This estimate excludes non-layer overhead (KV cache operations, Python dispatch, CUDA launch latency), which is empirically negligible: on the RTX 3090, the measured preloading time of 3.12 s differs from the estimated $C^{\text{opt}} = 3.13$ s by only 0.3%.

B. Swapping Overhead in Existing Methods

For GPUs such as the RTX 5070Ti that cannot preload all model parameters, an efficient parameter swapping strategy that minimizes the additional overhead is essential.

When inferring a model that exceeds GPU VRAM without explicit swapping, the behavior differs by operating system. The WDDM driver used in Windows and WSL2 environments transparently migrates data between VRAM and system RAM in 4KB–2MB page granularity through GPU page faults. As a result, Alpamayo-R1-10B inference takes 273.79 s on an RTX 3080Ti (12 GB) and 69.6 s on an RTX 5070 Ti (16GB). The KMD driver on native Linux raises an OOM error instead of performing transparent swapping, making an explicit offloading framework mandatory.

The Hugging Face Accelerate library [9] provides the blind swapping method. This mechanism statically partitions each parameter tensor across GPU and CPU at model load time, then synchronously transfers tensors to the GPU via forward hooks during inference, deleting them afterward. This approach has three fundamental inefficiencies:

- **Per-parameter transfer:** In a transformer layer, 14 weight tensors are transferred via individual `.to("cuda")` calls, resulting in 14 malloc-H2D operations per layer.
- **Synchronous execution:** The CPU thread blocks until each parameter arrives. The H2D transfer of layer $i+1$ cannot begin until the forward pass of layer i completes.
- **Global synchronization:** Module deletion triggers `empty_cache()` or block reclamation, causing global CUDA stream synchronization.

On the RTX 5070Ti, Hugging Face Accelerate offloading incurs an inference latency of 14.52 s, which substantially exceeds the theoretical lower bound. Moreover, this gap widens drastically as the available VRAM is further constrained (Fig. 2).

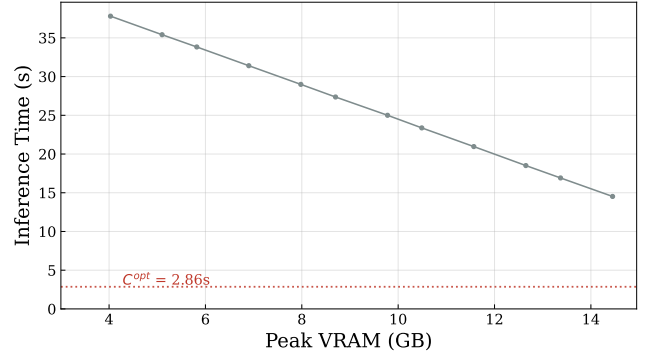


Fig. 2: VRAM–latency tradeoff of Hugging Face Accelerate offloading under varying VRAM limits on RTX 5070Ti.

TABLE III: Per-layer DMA and execution cost (RTX 5070 Ti, Sequential Demand Layering, 30 iterations, pinned memory, PCIe Gen5).

Module	C_{DMA} (ms)	C_{EXE} (ms)	r
Vision	0.9	8.3	0.11
VLM Prefill	10.9	16.6	0.66
VLM Decode	10.9	0.9	12.1
Diffusion	3.6	1.0	3.5

C. DMA Hiding Limitation

The primary source of swapping overhead is the transfer time for parameter data. Since transfers occur from CPU memory to GPU VRAM over PCIe, this overhead is determined by PCIe transfer performance—a hardware limitation. Applying pipelining with Demand Layering can hide this transfer time when a layer’s execution time exceeds its parameter transfer time. However, in Alpamayo, some modules have parameter transfer times that are much larger than their layer execution times (Table III). We define the DMA-to-EXE ratio $r = C_{\text{DMA}}/C_{\text{EXE}}$ for each module (Table III). Modules with $r < 1$ are EXE-intensive (DMA fully hideable), while $r > 1$ indicates DMA-intensive behavior where pipelining alone cannot eliminate the transfer overhead. In such cases, hiding the transfer time is structurally impossible, and selecting GPU-resident layers from the remaining VRAM to eliminate transfer time entirely becomes essential.

IV. SYSTEM ARCHITECTURE

A. Design Overview

Our work aims to minimize inference latency on VRAM-constrained commodity GPUs, approaching the theoretical lower bound as closely as possible. The theoretical lower bound can be measured directly on GPUs with sufficient VRAM by running Alpamayo inference with all parameters preloaded. For GPUs such as the RTX 5070Ti where preloading is infeasible due to VRAM constraints, the bound is estimated as the sum of per-layer execution times (Eq. (3)), since non-layer overhead is negligible.

To approach this theoretical lower bound (Table II), we proceed in three optimization stages:

- 1) **Sequential Demand Layering:** Reduces VRAM usage from model-level to layer-level by loading parameters on demand.
- 2) **Pipelined Demand Layering:** Hides parameter transfer time within layer execution time using transfer–compute overlap.
- 3) **GPU-Resident Layer Selection:** Allocates the freed VRAM to keep selected layers GPU-resident, eliminating the transfer time that pipelining could not fully hide.

B. Sequential Demand Layering

Demand Layering [7] loads each layer’s parameters immediately before its inference, as opposed to the conventional preloading approach that uploads all layer parameters to VRAM before the first inference cycle. While Demand Layering originally targeted CNNs with their sequential execution characteristics, Alpamayo is similarly well suited for this technique because parameters dominate its total VRAM consumption.

The most significant contribution of Demand Layering [7] was reducing parameter memory from model-level to layer-level granularity. Our framework likewise introduces per-layer on-demand parameter loading, reducing VRAM from 14.45 GB to 3.99 GB—a 72% reduction. By addressing the first inefficiency of Hugging Face Accelerate offloading—per-variable sequential allocation and loading within each layer—we consolidate parameter transfers at layer granularity. Despite the structural limitation that parameter transfer and layer execution remain serialized in this sequential variant, execution time is reduced from 14.52 s to 12.72 s.

C. Pipelined Demand Layering

GPUs possess both an Execution Engine and a Copy Engine, enabling concurrent layer inference (EXE) and parameter copying from CPU memory to GPU VRAM (DMA). We exploit this structural characteristic by applying pipelining to Demand Layering.

The purpose of pipelining is to hide parameter loading within layer execution time. We classify layers based on whether the parameter transfer time or the execution time dominates:

- **EXE-intensive modules:** $C_{\text{DMA}} < C_{\text{EXE}}$. DMA is fully hidden within EXE; overhead relative to preloading is a single C_{DMA} for the first layer.
- **DMA-intensive modules:** $C_{\text{DMA}} > C_{\text{EXE}}$. DMA cannot be fully hidden; each layer incurs overhead of $C_{\text{DMA}} - C_{\text{EXE}}$, yielding total overhead of $L \cdot (C_{\text{DMA}} - C_{\text{EXE}})$.

The CNNs targeted by Demand Layering [7] have EXE-intensive module behavior, enabling substantial memory savings at the cost of only a single C_{DMA} overhead. However, Alpamayo’s modules exhibit heterogeneous characteristics. We measured per-layer parameter transfer times and execution times for each Alpamayo module (Fig. 3). The results show

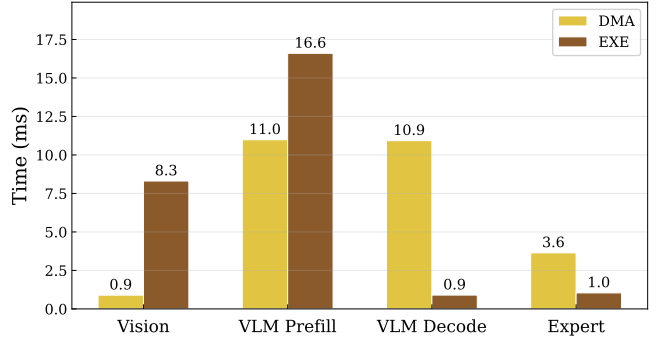


Fig. 3: Average per-layer DMA and execution times for each Alpamayo module (RTX 5070 Ti). ViT and VLM Prefill are EXE-intensive ($C_{\text{DMA}} < C_{\text{EXE}}$); VLM Decode and Expert are DMA-intensive ($C_{\text{DMA}} > C_{\text{EXE}}$).

that the ViT module and the VLM prefill phase are EXE-intensive, while the VLM decode phase and the Diffusion module are DMA-intensive.

DMA for the ViT module and the VLM prefill phase can be fully hidden by adjusting pipelining, and inference time can approach the theoretical lower bound. However, DMA for the VLM decode phase and the Diffusion module cannot be fully hidden. Thus, additional measures are required to reduce latency.

To implement pipelined Demand Layering, we introduce the *Double Flat Buffer* (DFB), a GPU memory management scheme that enables concurrent DMA transfer and layer execution without per-layer memory allocation overhead. The DFB comprises three design elements: (i) Buffer structure, (ii) Slot assignment and synchronization, and (iii) Parameter pinning.

(i) Buffer structure. The DFB allocates two contiguous GPU buffers (slots 0 and 1), each sized to hold the largest layer’s parameters across all modules. Rather than allocating and freeing GPU memory for each layer via `torch.empty()` and `empty_cache()`, all layers share the same two pre-allocated buffers in an alternating fashion. This eliminates the `empty_cache()` synchronization overhead that dominates Hugging Face Accelerate offloading.

All modules (ViT, VLM, Expert) share a single unified DFB instance, with each buffer slot sized to the largest layer across all modules. The total buffer memory is $2 \times M_{\text{max}}$, where M_{max} is the largest layer size among all modules.

(ii) Slot assignment and synchronization. Layers are assigned to slots via $s = i \bmod 2$, where i is the layer index. Two CUDA events govern the pipeline:

- **DMA-done event:** Recorded on the prefetch stream after H2D copy completes. The default (compute) stream waits on this event before executing the layer.
- **Compute-done event:** Recorded on the default stream after layer execution completes. The prefetch stream waits on this event before reusing the slot for the next layer’s DMA.

This two-event protocol ensures that (1) a layer never executes before its parameters arrive, and (2) a buffer slot

is never overwritten while its data is still in use. The prefetch stream submits the DMA for layer $i+1$ while the default stream executes layer i , achieving the transfer–compute overlap illustrated in Fig. 1(a)–(b).

(iii) Parameter pinning. All CPU-side layer parameters are registered as CUDA pinned (page-locked) memory via `pin_memory()`. Pinned memory enables the GPU’s Copy Engine to perform DMA transfers via `non_blocking=True` without CPU involvement, which is essential for achieving full CE/EE overlap. On the RTX 5070Ti (PCIe Gen5), pinned H2D throughput reaches 33.4 GB/s.

D. GPU-Resident Layer Decision Policy

Sequential Demand Layering reduces VRAM usage to layer-level granularity, and pipelined Demand Layering hides transfer time for EXE-intensive modules. However, DMA-intensive modules cannot achieve full transfer hiding. The *GPU-resident layer decision policy* addresses this limitation: keeping a specific layer GPU-resident eliminates its DMA time at the cost of permanently occupying VRAM equal to that layer’s parameter size.

The efficiency of GPU residency varies across modules. We define the *residency benefit* of a layer as the inference time saved by keeping it resident, divided by its memory footprint. For modules whose layers are invoked repeatedly—e.g., VLM decode layers are invoked once per output token during autoregressive generation, and diffusion layers are invoked once per diffusion step—the time savings are multiplied by the repetition count:

$$B_\ell = \frac{R_\ell \cdot \Delta C_\ell}{M_\ell}. \quad (4)$$

where B_ℓ is the residency benefit of layer ℓ , R_ℓ is its repetition count, $\Delta C_\ell = C_{\text{DMA}}^\ell - \min(C_{\text{DMA}}^\ell, C_{\text{EXE}}^\ell)$ is the per-invocation time saved, and M_ℓ is its memory footprint.

Before computing each module’s residency benefit, we measured per-layer DMA and execution times and confirmed that layers within the same module are functionally identical in code structure and exhibit identical measured timings (Fig. 3). We now derive the residency benefit based on each module’s characteristics, and then apply it to Alpamayo’s specific modules.

EXE-Intensive Modules. Consider a module with L layers where $C_{\text{DMA}} < C_{\text{EXE}}$, invoked R times per inference. Pipelining fully hides all DMA transfers except the initial one per invocation, yielding:

$$C_{\text{EXE-int}} = R \cdot (C_{\text{DMA}} + L \cdot C_{\text{EXE}}). \quad (5)$$

Fig. 4 illustrates this pipeline: since $C_{\text{EXE}} > C_{\text{DMA}}$, removing a middle layer’s DMA (e.g., D_2) leaves an idle slot without reducing total time. Keeping the first layer resident removes the initial DMA across all R invocations in Eq. (5). Keeping any middle or last layer resident provides no benefit, as its DMA is already hidden within the preceding layer’s EXE. Table IV summarizes the residency benefit by position.

TABLE IV: EXE-intensive module (R repetitions): residency benefit by position.

Position	ΔC	Benefit B
First	$R \cdot C_{\text{DMA}}$	$R \cdot C_{\text{DMA}}/M$
Middle	0	0
Last	0	0

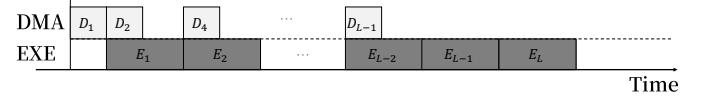


Fig. 4: Pipelined EXE-intensive module with layer 3 resident. Removing D_2 leaves an idle DMA slot, but total time is unchanged since EXE dominates—confirming zero benefit for middle-layer residency.

DMA-Intensive Modules. Consider a module with L layers where $C_{\text{DMA}} > C_{\text{EXE}}$, invoked R times per inference (e.g., once per output token or per diffusion step). Under pipelining, all L layers’ DMA transfers form the bottleneck, and only the last layer’s execution cannot be overlapped (the “trailing EXE”). The total execution time is therefore:

$$C_{\text{DMA-int}} = R \cdot (L \cdot C_{\text{DMA}} + C_{\text{EXE}}), \quad (6)$$

where the C_{EXE} term is the single trailing execution of the last layer per invocation (Fig. 5). This applies to both VLM Decode ($R=N$ tokens, $L=36$) and the Diffusion Expert ($R=10$ steps, $L=36$).

Keeping a layer resident eliminates its C_{DMA} across all R invocations in Eq. (6). For the last layer, the benefit is reduced because DMA of the next iteration’s first layer can partially overlap with its EXE. The residency benefit by position is summarized in Table V.

When consecutive layers are kept resident, the DMA interval for the next offloaded layer can overlap with the execution of multiple resident layers. The maximum number of consecutive resident layers whose EXE can be hidden within a single DMA is $\lfloor C_{\text{DMA}}/C_{\text{EXE}} \rfloor$; we call this the *consecutive residency limit*.

Hybrid Modules. When the same parameters serve both an EXE-intensive phase and a DMA-intensive phase, keeping a layer resident benefits both. Since VLM Prefill is EXE-intensive ($C_{\text{DMA}} < C_{\text{prefill}}$) and VLM Decode is DMA-intensive ($C_{\text{DMA}} > C_{\text{decode}}$), each phase uses its respective pipelined formula (Eq. (1) for prefill, Eq. (2) for decode): leading DMA + all EXE for prefill, and all DMA + trailing EXE for decode. Let the DMA-intensive phase repeat N times. The full-offload execution time is:

$$C_{\text{hybrid}} = C_{\text{DMA}} + L \cdot C_{\text{prefill}} + N \cdot (L \cdot C_{\text{DMA}} + C_{\text{decode}}). \quad (7)$$

The combined benefit is the sum of the EXE-intensive first-layer savings and the DMA-intensive repeated savings, as shown in Table VI.

The last layer’s reduced benefit warrants explanation: in the full-offload pipeline, the last layer’s execution serves as the

TABLE V: DMA-intensive module (R repetitions): residency benefit by position.

Position	ΔC	Benefit B
First / Middle	$R \cdot C_{\text{DMA}}$	$R \cdot C_{\text{DMA}}/M$
Last	$R \cdot (C_{\text{DMA}} - C_{\text{EXE}})$	$R \cdot (C_{\text{DMA}} - C_{\text{EXE}})/M$

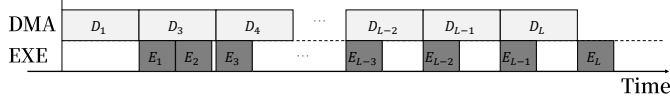


Fig. 5: Pipelined DMA-intensive module with layer 2 resident. DMA dominates the timeline ($C_{\text{DMA}} > C_{\text{EXE}}$), and each layer incurs an irreducible overhead of $C_{\text{DMA}} - C_{\text{EXE}}$ that pipelining cannot hide.

“trailing EXE” that is already excluded from the pipelined DMA chain (the C_{decode} term in Eq. (7)). When the last layer becomes resident and its DMA is eliminated, the *penultimate* layer becomes the new pipeline tail, introducing a new trailing execution cost of C_{decode} that was previously absorbed. Consequently, the net savings is only $N \cdot (C_{\text{DMA}} - C_{\text{decode}})$ rather than the full $N \cdot C_{\text{DMA}}$.

We now apply this framework to Alpamayo’s modules. We instantiate the general framework for each Alpamayo module using the profiling data from Table III. Non-layer components (Patch Embed, Patch Merger, LM-Head) have higher per-MB benefit than any layer and are kept resident in all configurations.

- **ViT Encoder** (27 layers, $M=29.1$ MB): EXE-intensive ($r=0.11$, $R=1$). $B_{\text{first}} = 1 \times 0.9/29.1 = 0.031$ ms/MB; $B_{\text{mid}} = B_{\text{last}} = 0$.
- **VLM Backbone** (36 layers, $M=368.0$ MB): Hybrid. Pre-fill is EXE-intensive ($r=0.66$); Decode is DMA-intensive ($r=12.1$, $N=21$ tokens). We fix $N=21$ throughout our analysis, matching the reference evaluation clip; this is a conservative choice, as the empirically measured median is 15 tokens (Fig. 7). $B_{\text{first}} = 22 \times 10.9/368.0 = 0.651$ ms/MB; $B_{\text{mid}} = 21 \times 10.9/368.0 = 0.622$ ms/MB; $B_{\text{last}} = 21 \times (10.9 - 0.9)/368.0 = 0.571$ ms/MB.
- **Expert Decoder** (36 layers, $M=120.8$ MB): DMA-intensive ($r=3.5$, $R=10$ steps). $B_{\text{first}} = B_{\text{mid}} = 10 \times 3.6/120.8 = 0.298$ ms/MB; $B_{\text{last}} = 10 \times (3.6 - 1.0)/120.8 = 0.215$ ms/MB.

Table VII summarizes the computed benefits. VLM layers dominate across all positions, confirming that optimal residency allocation should prioritize VLM layers.

Fig. 6 shows the residency benefit of each module’s layers as a function of the output token count N with diffusion steps fixed at 10. Computing the crossover points reveals that VLM residency benefit surpasses ViT and Diffusion benefit when the output token count exceeds certain thresholds.

Fig. 7 shows the distribution of output token counts across 100 randomly sampled clips. The median is 15 tokens with P5–P95 range of 12–21, confirming that VLM residency benefit dominates in virtually all driving scenarios. The empirical

TABLE VI: Hybrid module (N decode repetitions): residency benefit by position.

Position	ΔC	Benefit B
First	$(N+1) \cdot C_{\text{DMA}}$	$(N+1) \cdot C_{\text{DMA}}/M$
Middle	$N \cdot C_{\text{DMA}}$	$N \cdot C_{\text{DMA}}/M$
Last	$N \cdot (C_{\text{DMA}} - C_{\text{decode}})$	$N \cdot (C_{\text{DMA}} - C_{\text{decode}})/M$

TABLE VII: Computed residency benefit for each Alpamayo module (RTX 5070 Ti, $N=21$).

Module	Type	B_{first}	B_{mid} (ms/MB)	B_{last}
ViT	EXE-int	0.031	0	0
VLM	Hybrid	0.651	0.622	0.571
Expert	DMA-int	0.298	0.298	0.215

distribution confirms that ViT and Diffusion layers have lower residency benefit than VLM layers in the vast majority of cases. Furthermore, on the RTX 5070 Ti, VRAM is insufficient to keep all VLM layers resident. We therefore conclude that ViT and Diffusion layers should always be offloaded, and optimal residency allocation should focus exclusively on VLM layers.

Fig. 8 shows inference time as a function of the number of GPU-resident VLM layers, comparing configurations where ViT and Diffusion modules are kept resident versus offloaded. Inference time decreases linearly with the number of resident VLM layers. Consistent with the residency benefit analysis, offloading ViT and Diffusion modules yields superior inference performance for the same VRAM budget, confirming the theoretical prediction experimentally.

Having confirmed the residency benefit experimentally, we now determine the optimal resident layer configuration. Naïvely, selecting which layers to keep resident from 36 VLM layers requires evaluating 2^{36} possible configurations. However, since all layers within the same module are functionally identical and exhibit uniform DMA and execution times, the residency benefit depends only on the *number* of resident layers, not on their specific identities. This reduces the search space from 2^{36} to at most 37 configurations (0 through 36 resident layers).

The remaining question is *which* k layers to select when keeping k layers resident. Three observations guide this decision:

- 1) **First layer inclusion:** The first layer provides strictly higher benefit than middle layers ($(N+1) \cdot C_{\text{DMA}}$ vs. $N \cdot C_{\text{DMA}}$), so it is always included.
- 2) **Last layer exclusion:** The last layer provides the lowest benefit ($N \cdot (C_{\text{DMA}} - C_{\text{decode}})$), so it is always excluded.
- 3) **Consecutive residency limit:** As discussed in Section IV-D, when more than $\lfloor C_{\text{DMA}}/C_{\text{decode}} \rfloor \approx 12$ consecutive layers are kept resident, the DMA interval of the next offloaded layer can no longer fully overlap with the execution of the resident layers, causing a marginal loss in residency benefit.

While consecutive and interleaved placement yield similar

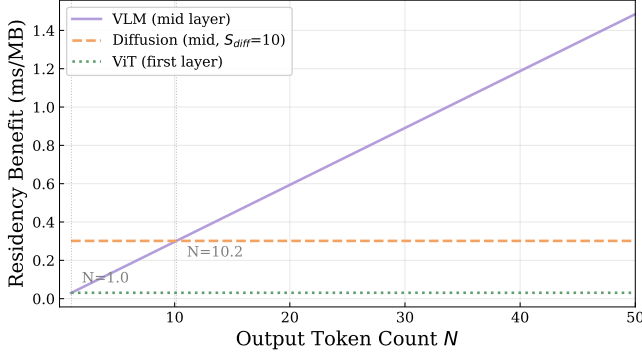


Fig. 6: Residency benefit per module versus output token count ($S_{\text{diff}}=10$). VLM benefit exceeds Diffusion at $N>10$ and dominates at Alpamayo’s typical $N=21$.

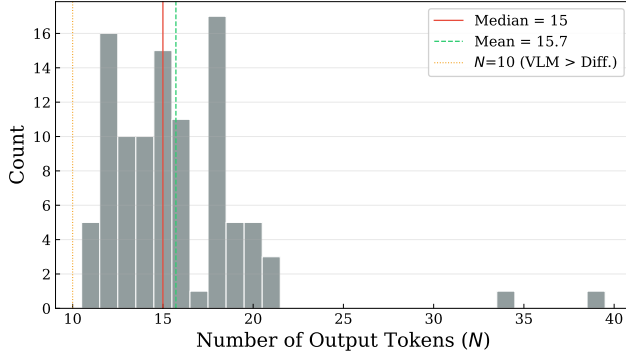


Fig. 7: Distribution of VLM output token counts across 100 randomly sampled driving clips. All inputs produce $N \geq 11$ tokens (median=15, mean=15.7), well above the $N=10$ threshold where VLM residency benefit exceeds Diffusion.

overall performance, interleaved placement avoids the consecutive residency limit entirely. We therefore adopt the following *interleaved residency placement* policy: given k layers to keep resident among layers 0 through 34 (first included, last excluded), the resident layers are placed at evenly spaced indices to maximize the gap between consecutive resident layers. Formally, the k resident layer indices are:

$$\mathcal{R}(k) = \left\{ \left\lfloor \frac{i \cdot 35}{k} \right\rfloor \mid i = 0, 1, \dots, k-1 \right\}. \quad (8)$$

Eq. (8) ensures that the inter-resident gap remains at least $\lceil 35/k \rceil$, which stays above the consecutive residency limit for all practical k values on the RTX 5070Ti. Combined with the performance prediction model, this enables our method to determine the optimal configuration—both the number and placement of resident layers—from a single profiling run without exhaustive search.

E. Performance Prediction Model

The residency benefit B_ℓ defined in Eq. (4) directly yields a performance prediction model. Since inference time decreases linearly with the number of resident VLM layers, the entire

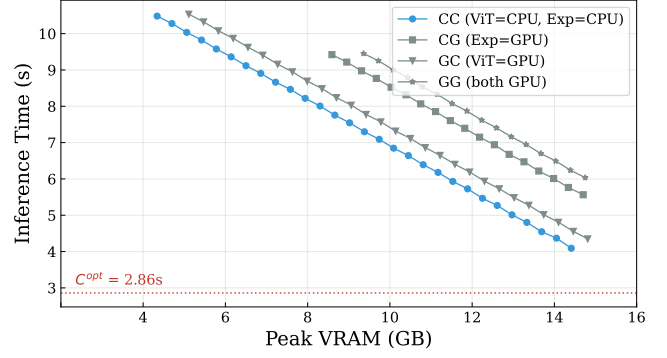


Fig. 8: Module residency comparison. Keeping ViT and Expert offloaded (CC) yields the lowest latency for any given VRAM budget, confirming the theoretical residency benefit analysis.

VRAM–latency trade-off curve can be predicted from profiling data and a single measurement.

Prediction Model. Since the first layer is always resident and the last layer is always excluded, the k additional resident layers are all middle layers with uniform benefit $B_{\text{VLM}} = B_{\text{mid}}$. Given B_{VLM} and the memory footprint M_{VLM} of a single VLM layer, the inference time with k GPU-resident VLM layers is:

$$C_{\text{total}}(k) = C_{\text{total}}(0) - k \cdot B_{\text{VLM}} \cdot M_{\text{VLM}}, \quad (9)$$

where $C_{\text{total}}(0)$ is the full-offload inference time measured on the final system. Equivalently, for a given VRAM budget V_{resident} allocated to resident layers:

$$C_{\text{total}}(V) = C_{\text{total}}(0) - B_{\text{VLM}} \cdot V_{\text{resident}}.$$

The slope of the linear model in Eq. (9) is $-B_{\text{VLM}} \cdot M_{\text{VLM}}$, which is computed entirely from profiling data (C_{DMA} , C_{EXE} , repetition count, and layer size). Only $C_{\text{total}}(0)$ —the intercept—requires a single measurement on the final system. This enables prediction of the complete VRAM–latency trade-off curve without exhaustive experimentation.

On the RTX 5070 Ti, the theoretical slope from profiling is -229.5 ms/layer ; the measured slope from linear regression over 29 configurations is -228.0 ms/layer , a difference of 0.7%. The residual gap is attributable to implementation-level overhead (Python hook dispatch, CUDA event synchronization) that is eliminated when a layer becomes resident, and is not a fundamental limitation of the prediction model.

Validation. Table VIII compares the predicted and measured inference times across the full resident sweep range. The prediction error remains within 1.3% across all configurations, confirming that a single profiling run and one measurement suffice to predict the entire VRAM–latency trade-off curve.

V. EVALUATION

A. Experimental Setup

All experiments use NVIDIA’s Alpamayo-R1-10B (21.52 GB, BF16) with 10 diffusion steps. Each configuration is measured over 30 trials with one warmup run; we report the

TABLE VIII: Predicted vs. measured inference time for varying numbers of GPU-resident VLM layers (RTX 5070 Ti).

Resident	Predicted (s)	Measured (s)	Error
0	10.482	10.482	$\pm 0.00\%$
5	9.335	9.360	-0.27%
10	8.187	8.219	-0.39%
15	7.040	7.090	-0.72%
20	5.892	5.932	-0.66%
25	4.745	4.803	-1.22%
28	4.056	4.092	-0.87%

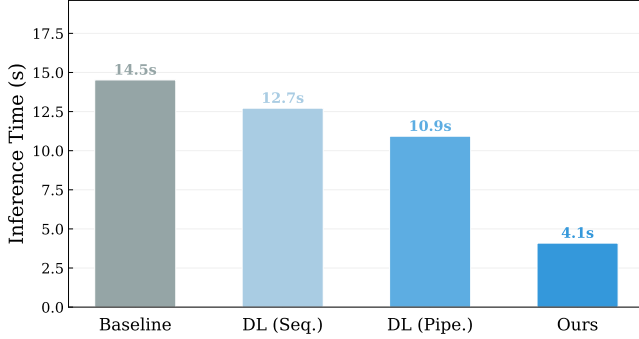


Fig. 9: Progressive optimization: Baseline $\rightarrow$ Sequential DL $\rightarrow$ Pipelined DL $\rightarrow$ Ours ($N_r=28$). Each stage improves latency; resident layer selection provides the largest gain ($2.67\times$).

mean after discarding the first trial. GPU clocks are locked at their maximum frequency via `nvidia-smi -lgc` to eliminate dynamic frequency scaling. All host-side parameter tensors use pinned memory (`pin_memory()`) for optimal DMA throughput.

Hardware. The primary evaluation platform is the RTX 5070Ti (16 GB VRAM, PCIe Gen5 x16, DDR5-5600 single-channel) running Ubuntu with Linux kernel 6.8. Cross-platform validation uses the RTX 3080Ti (12 GB VRAM, PCIe Gen3 x16).

Baselines. We compare against two baselines: (1) Hugging Face Accelerate’s `device_map="auto"` [9] with a 15 GB VRAM limit (14.52 s, 14.45 GB peak), representing the standard practitioner approach, and (2) the preloading baseline estimated as $C^{\text{opt}} = 2.86$ s on the RTX 5070 Ti and $C^{\text{opt}} = 5.86$ s on the RTX 3080 Ti.

B. Progressive Optimization

Fig. 9 shows the inference time reduction across the three optimization stages of our method. Sequential Demand Layering reduces inference time from 14.52 s to 12.72 s ($1.14\times$) while cutting VRAM from 14.45 GB to 3.99 GB—a 72% reduction. Pipelined Demand Layering with a Double Flat Buffer further reduces time to 10.93 s ($1.33\times$) by hiding DMA transfers within EXE-intensive module execution. Finally, with 28 GPU-resident VLM layers ($N_r=28$), our method achieves 4.09 s ($3.55\times$), approaching the theoretical lower bound of 2.86 s.

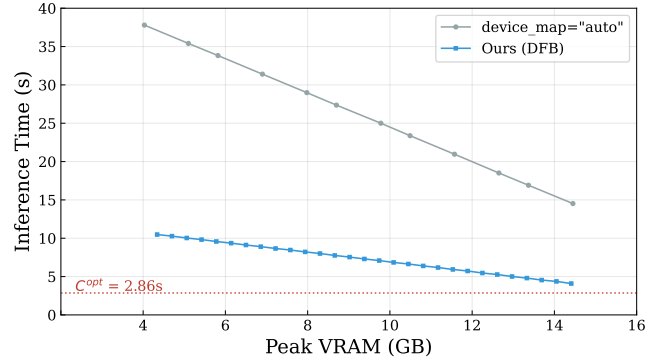


Fig. 10: VRAM–latency tradeoff. Our method consistently achieves $3.5\text{--}3.6\times$ lower latency than Accelerate offloading at every VRAM budget.

TABLE IX: Numerical correctness: max absolute trajectory error between baseline (full GPU) and DFB at varying N_r (RTX 3090).

N_r	Max Abs. Error	Bit-exact
0	0.0	✓
5	0.0	✓
10	0.0	✓
15	0.0	✓
20	0.0	✓

C. VRAM–Latency Tradeoff

Fig. 10 compares the VRAM–latency tradeoff of our method against Accelerate offloading across the full operating range. At every VRAM budget, our method achieves $3.5\text{--}3.6\times$ lower latency than the baseline. Notably, Accelerate offloading requires 35.41 s at 5 GB VRAM, while our method achieves 10.48 s at 4.3 GB, corresponding to $3.4\times$ lower latency. At the maximum configuration ($N_r=28$, 14.41 GB), our method reaches 4.09 s, which is $3.55\times$ faster than the baseline at comparable VRAM.

D. Numerical Correctness

Table IX verifies that Demand Layering with Double Flat Buffer produces bit-exact output regardless of the number of resident layers. We compare the predicted trajectory (64 waypoints $\times$ 3 coordinates) against the baseline (full GPU preloading) on the RTX 3090. The maximum absolute error is zero across all configurations, confirming that our optimization introduces no numerical degradation.

E. Cross-Platform and Cross-Model Scalability

To verify that our analysis framework generalizes beyond a single GPU and model, we evaluate on an RTX 3080 Ti (12 GB VRAM, PCIe Gen3 x16) with both Alpamayo-R1-10B and OpenVLA-7B.

Cross-Platform Profiling. Table X compares the per-module DMA/EXE ratio r between the RTX 5070Ti (PCIe Gen5) and RTX 3080 Ti (PCIe Gen3).

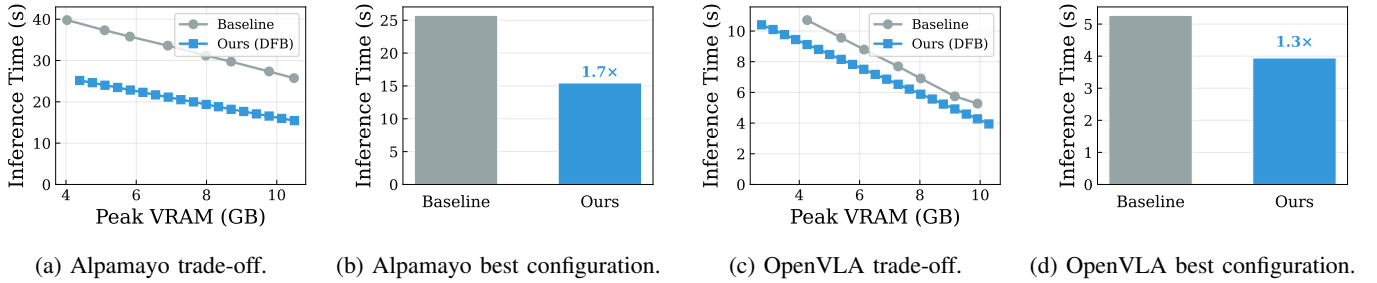


Fig. 11: Comparison of baseline and DFB on RTX 3080 Ti.

TABLE X: Cross-platform DMA/EXE ratio comparison. Lower PCIe bandwidth shifts more modules toward DMA-intensive.

Module	RTX 5070 Ti (Gen5)		RTX 3080 Ti (Gen3)	
	C_{DMA}	r	C_{DMA}	r
ViT	0.9 ms	0.1	2.5 ms	0.24
VLM Prefill	10.9 ms	0.7	30.8 ms	1.5
VLM Decode	10.9 ms	12.1	30.8 ms	20.5
Expert	3.6 ms	3.5	10.1 ms	6.7

The EXE-intensive vs. DMA-intensive classification is consistent across both platforms: ViT remains EXE-intensive ($r < 1$), while VLM Decode and Expert are DMA-intensive ($r \gg 1$) on both GPUs. Notably, on the RTX 3080 Ti with PCIe Gen3, VLM Prefill transitions from EXE-intensive ($r=0.7$) to DMA-intensive ($r=1.5$), demonstrating that lower PCIe bandwidth shifts more modules toward DMA-intensive behavior, making GPU-resident layer selection even more critical on older platforms.

Fig. 11(a)–(b) show the Alpamayo results on the RTX 3080 Ti. At comparable VRAM (~ 10.5 GB), our method achieves 15.46 s ($N_r=17$) versus the Accelerate baseline’s 25.74 s—a $1.7\times$ speedup, confirming that the DFB framework transfers effectively to VRAM-constrained PCIe Gen3 platforms.

Cross-Model Validation: OpenVLA-7B. To verify model generality, we apply our framework to OpenVLA-7B [10], a 7B-parameter VLA model with 32 LLM layers. Fig. 11(c)–(d) show the results on the RTX 3080 Ti. At comparable VRAM (~ 10 GB), our method achieves 3.94 s ($N_r=20$) versus the Accelerate baseline’s 5.27 s—a $1.3\times$ speedup, demonstrating that the DFB framework and residency policy generalize across different VLA architectures.

The lower speedup on the RTX 3080 Ti compared to the RTX 5070 Ti ($1.7\times$ vs. $3.55\times$ for Alpamayo) is expected: PCIe Gen3 bandwidth is roughly $3\times$ lower than Gen5, increasing C_{DMA} and the DMA-intensive bottleneck that our method targets. Since the baseline also suffers from higher transfer overhead on Gen3, the relative gap narrows, but our method still consistently outperforms the baseline across all VRAM budgets on both platforms.

VI. DISCUSSION

A. Platform-Specific Issues

During experimentation, we discovered that Linux kernel 6.17 on the Arrow Lake platform causes CPU memory bandwidth to collapse to 3.6% of theoretical throughput, inflating inference time to 59.09 s. Downgrading to kernel 6.8 resolved the issue (inference time reduced to 8.95 s, a $6.6\times$ improvement), underscoring that system-level optimization is sensitive to OS and driver layers. All experiments specify the exact kernel version for reproducibility.

VII. RELATED WORK

Our work lies at the intersection of GPU memory management, model offloading, and real-time DNN inference optimization. We organize related work into three categories.

A. Layer-wise Offloading and Memory Virtualization

vDNN [11] pioneered GPU memory virtualization by dynamically swapping activation data between GPU and CPU during training, achieving up to 95% memory reduction. While it established the prefetch-overlap paradigm that our work also employs, vDNN targets training activations rather than inference weights, differing in both data type and execution pattern.

Demand Layering [7] is the most directly related prior work. It exploits the layer-by-layer execution pattern of DNNs to load parameters from SSD to GPU one layer at a time, achieving 96.5% memory reduction with 14.8% latency overhead through pipelined I/O hiding. However, the CNNs targeted by Demand Layering are predominantly EXE-intensive ($C_{DMA} < C_{EXE}$), allowing pipelining to hide most of the additional transfer delays. In contrast, VLA models such as Alpamayo contain DMA-intensive modules ($C_{DMA} \gg C_{EXE}$, $r \approx 12$ for VLM Decode), where pipelining alone cannot hide the transfer overhead. Our work analyzes these heterogeneous module characteristics and introduces a residency policy to address the DMA-intensive bottleneck.

RT-Swap [12] provides transparent GPU memory swap scheduling for multi-DNN real-time inference, making 72% more task sets schedulable when memory demand exceeds capacity by 96.2%. While RT-Swap manages memory at the task level for concurrent DNNs, our work manages memory at the layer level within a single large VLA model.

B. Large Model Inference Offloading

DeepSpeed Inference [13] supports multi-GPU Transformer inference through tensor parallelism and model-parallel inference. Its ZeRO-Inference component provides layer-wise CPU/NVMe offloading with prefetch, but optimizes for throughput with large batch sizes rather than single-request latency for real-time control.

FlexGen [14] aggregates GPU, CPU, and disk memory to run LLMs on a single GPU, using linear programming to optimize tensor placement and achieving $100\times$ throughput improvement for OPT-175B. However, FlexGen targets batch throughput optimization, which diverges from the single-inference latency minimization required for real-time VLA deployment.

NEO [15] offloads attention computation and KV cache from GPU to CPU via asymmetric pipelining, achieving up to $7.5\times$ throughput improvement on T4 GPUs. Unlike our approach which offloads layer weights, NEO offloads computation itself and targets uniform Transformer stacks, whereas our VLA model comprises a heterogeneous multi-stage pipeline (vision encoder, language model, and diffusion model).

PowerInfer [16] exploits neuron activation sparsity in LLMs to enable GPU-CPU hybrid inference, selectively loading only “hot” neurons onto the GPU. While highly effective for sparse-activation LLMs, VLA models like Alpamayo activate all parameters during inference (dense execution). Therefore direct adoption is non-trivial.

llama.cpp [17] is a widely deployed CPU/GPU offloading framework for LLM inference on consumer hardware. It supports partial GPU offloading at layer granularity but does not explicitly provide the prefetching/residency mechanisms targeted in this work.

Hugging Face Accelerate [9] provides an automatic device-mapping option (`device_map="auto"`) for static parameter partitioning across GPU/CPU at load time. However, it does not explicitly expose the prefetching and transfer-compute overlap mechanisms used in our method, resulting in significant performance degradation under VRAM constraints as demonstrated in our analysis (Section III).

C. Model Compression

GPTQ [5] and AWQ [6] compress LLM weights to typically 3–4 bits via post-training quantization with minimal accuracy loss. These approaches trade model precision for memory savings—an orthogonal direction to our system-level optimization that preserves full BF16 precision. While quantization and our techniques are combinable in principle, this work demonstrates the performance achievable through system optimization alone.

VIII. CONCLUSION

We presented a framework for memory-efficient VLA model inference on VRAM-constrained commodity GPUs without model modification. Applied to the 21.52 GB Alpamayo-R1-10B model on an RTX 5070 Ti (16 GB), the framework

demonstrates that system-level optimization alone can bridge the memory gap for practical VLA deployment.

Our key contributions are fourfold. First, we characterized the heterogeneous module behavior of VLA models, classifying layers as EXE-intensive or DMA-intensive and revealing that VLM Decode layers ($r \approx 12$) make transfer hiding via pipelining structurally impossible. Second, we derived closed-form residency benefit expressions for EXE-intensive, DMA-intensive, and hybrid modules, and proposed an interleaved placement policy that reduces the 2^{36} configuration search space to a single profiling run. Third, we presented a performance prediction model that estimates inference time for arbitrary residency configurations within 1.3% error from profiling data alone. Fourth, we demonstrated $3.55\times$ speedup over Accelerate offloading on the RTX 5070 Ti with full BF16 precision and validated the analysis framework across multiple GPU platforms.

We acknowledge that the achieved inference time of 4.09 s on the RTX 5070 Ti remains far from the sub-100 ms regime required for closed-loop autonomous driving control. However, this limitation is fundamental to current VLA model scale rather than to our optimization framework: even on the high-end RTX 5090 (32 GB) with full GPU preloading and no memory constraints, Alpamayo-R1-10B inference still requires 1.03 s—an order of magnitude above real-time requirements. Bridging this gap requires advances in both model architecture (smaller, faster VLA models) and hardware (higher compute throughput), which is the fundamental challenge for VLA-based autonomous driving. Our contribution lies in ensuring that, for any given model and GPU, inference latency approaches the theoretical optimum C^{opt} as closely as possible under VRAM constraints.

REFERENCES

- [1] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, “End-to-end autonomous driving: Challenges and frontiers,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 10 164–10 183, 2024.
- [2] Y. Wang, W. Luo, J. Bai *et al.*, “Alpamayo-r1: Bridging reasoning and action prediction for generalizable autonomous driving in the long tail,” *arXiv preprint arXiv:2511.00088*, 2025.
- [3] A. Brohan, N. Brown, J. Carbajal *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv preprint arXiv:2307.15818*, 2023.
- [4] X. Tian, J. Gu, B. Li, Y. Liu, Y. Wang, Z. Zhao, K. Zhan, P. Jia, X. Lang, and H. Zhao, “DriveVLM: The convergence of autonomous driving and large vision-language models,” *arXiv preprint arXiv:2402.12289*, 2024.
- [5] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [6] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- [7] M. Ji, S. Yi, C. Koo, S. Ahn, D. Seo, N. Dutt, and J.-C. Kim, “Demand layering for real-time DNN inference with minimized memory usage,” in *Proceedings of the 43rd IEEE Real-Time Systems Symposium (RTSS)*, 2022, pp. 277–290.
- [8] NVIDIA, “Cuda c++ programming guide,” <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, 2026, cUDA Toolkit Documentation, accessed: 2026-03-26.

- [9] Hugging Face, “Accelerate,” <https://github.com/huggingface/accelerate>, 2026, gitHub repository, accessed: 2026-03-26.
- [10] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “Open-VLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [11] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfiqar, and S. W. Keckler, “vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design,” in *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016.
- [12] W. Kang, J. Lee, Y. Lee, S. Oh, K. Lee, and H. S. Chwa, “RT-Swap: Addressing gpu memory bottlenecks for real-time multi-dnn inference,” in *Proceedings of the 2024 IEEE 30th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2024, pp. 373–385.
- [13] R. Y. Aminabadi, S. Rajbhandari, M. Zhang, A. A. Awan, C. Li, D. Li, E. Zheng, J. Rasley, S. Smith, O. Ruwase, and Y. He, “DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale,” in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022, pp. 1–15.
- [14] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Re, I. Stoica, and C. Zhang, “Flexgen: High-throughput generative inference of large language models with a single gpu,” in *Proceedings of the 40th International Conference on Machine Learning*, 2023, pp. 31 094–31 116.
- [15] X. Jiang, Y. Zhou, S. Cao, I. Stoica, and M. Yu, “Neo: Saving gpu memory crisis with cpu offloading for online llm inference,” 2024.
- [16] Y. Song, Z. Mi, H. Xie, and H. Chen, “Powerinfer: Fast large language model serving with a consumer-grade gpu,” 2023.
- [17] G. Gerganov, “llama.cpp,” <https://github.com/ggml-org/llama.cpp>, 2026, gitHub repository, accessed: 2026-03-26.